PetCare

Application de gestion et de suivi des animaux domestiques

JALON 6

Déploiement & Mise en Production — Livrable Final

Projet	PetCare — Gestion du suivi animalier
Jalon	6 — Déploiement & Mise en Production
Échéance	30/06/2026
Auteur	Tristan DZIOCH
Formation	Concepteur Développeur d'Applications (CDA) — IPSSI
Version	v1.0 (Livrable Final)
Dépôt Git	https://github.com/Nastry69/PetCare (tag : `v1.0`)

Sommaire

Section	Intitulé
—	Introduction
1	Résultats finaux du projet
2	Infrastructure Docker (6 services)
3	Système d'emails (MailerService)
4	Rappels automatiques à l'heure exacte
5	Pipeline CI/CD GitHub Actions
6	Sécurité & conformité RGPD
7	Vérification de conformité au CDC technique
8	Documentation & guide d'utilisation
9	Bilan & perspectives
—	Conclusion

> Ce livrable se concentre sur le déploiement et la mise en production. Les chapitres de conception (cahier des charges, UI/UX, modélisation de la base, UML, sécurité détaillée) font l'objet de mes livrables des Jalons 1 à 5, disponibles dans le dossier `Docs/` du dépôt.

Introduction

Ce document constitue le **livrable final** de mon projet fil rouge PetCare, réalisé dans le cadre de ma formation Concepteur Développeur d'Applications (CDA).

J'ai conçu ce livrable pour couvrir le **Jalon 6**, dont l'objectif est de rendre mon application **déployable en une seule commande**, fiable en production, et de documenter l'ensemble des choix techniques que j'ai pris tout au long du projet.

Avec ce livrable, je finalise **PetCare v1.0** autour de cinq piliers que j'ai mis en place :

- une infrastructure **Docker complète de 6 services** dotée d'un entrypoint automatisé ;
- un système d'**emails transactionnels** avec 4 templates HTML responsive ;
- des **rappels automatiques** planifiés à la minute exacte du rendez-vous ;
- une pipeline **CI/CD GitHub Actions** (3 jobs, 100 % passing, scans de sécurité intégrés) ;
- une **documentation de déploiement** complète.

> **Nota.** Ce document est mon livrable final consolidé pour le Jalon 6. Il se concentre sur le déploiement et la mise en production (chapitre XI du rapport CDA) et s'appuie sur mes livrables précédents (Jalons 1 à 5), auxquels je renvoie pour les éléments déjà documentés (cahier des charges, conception UI/UX, modélisation de la base, UML, sécurité détaillée).

---

1. Résultats finaux du projet

1.1 — Couverture fonctionnelle complète

J'ai livré l'intégralité des **six fonctionnalités V1** que j'avais définies dans mon cahier des charges fonctionnel.

#	Fonctionnalité	Back	Front	Tests	Statut
F1	Gestion des utilisateurs (JWT, OAuth Google, profil, reset mot de passe)	✓	✓	✓	Livré
F2	Gestion des animaux (CRUD + upload de photos)	✓	✓	✓	Livré
F3	Suivi des événements et des soins	✓	✓	✓	Livré
F4	Rappels automatiques (heure exacte du RDV via Scheduler)	✓	✓	✓	Livré
F5	Partage de fiches animaux (rôles lecture / écriture)	✓	✓	✓	Livré
F6	Conformité RGPD (export des données + suppression du compte)	✓	✓	✓	Livré

1.2 — Chiffres clés du projet

Indicateur	Valeur
Endpoints API REST	33 routes exposées sous `/api`
Entités Doctrine	6 (User, Animal, Evenement, TypeEvenement, PartageAnimal, InvitationEnAttente)
Services métier	4 (AnimalService, EvenementService, PartageAnimalService, MailerService)
Templates email HTML responsive	4 (bienvenue, rappel, reset mot de passe, invitation)
Tests PHPUnit (backend)	167 (57 unitaires + 110 fonctionnels — 336 assertions)
Tests Vitest (frontend)	26 (AuthContext, Login, Register)
Services Docker	6 (app, nginx, frontend, database, worker, mailer)
Jobs CI GitHub Actions	3 (PHPUnit + sécurité, PHP Lint, Frontend build + sécurité)
Migrations Doctrine	6 (Version20260327 → Version20260607)
Version finale	v1.0 (tag Git de release)

---

2. Infrastructure Docker (6 services)

Ce volet correspond au chapitre XI — Déploiement et mise en production du rapport final CDA.

2.1 — Architecture Docker Compose

J'ai orchestré l'ensemble de l'application avec Docker Compose. Chaque responsabilité est isolée dans son propre conteneur, ce qui me garantit un environnement reproductible du développement à la production.

Service	Image	Rôle	Port
app	PHP 8.4-FPM Alpine	API Symfony (backend)	—
nginx	nginx:1.27-alpine	Reverse proxy → service app	8000
frontend	Node 22 → nginx	SPA React (build statique `dist/` servi par nginx)	3000
database	PostgreSQL 16	Base de données relationnelle	5432
worker	PHP 8.4-FPM Alpine	Consommateur Symfony Messenger (rappels)	—
mailer	Mailpit	SMTP de développement + UI web des emails	8025

2.2 — Lancement en une commande

J'ai fait en sorte que toute l'infrastructure se déploie en deux commandes seulement :

```
# 1. Récupérer le projet
git clone https://github.com/Nastry69/PetCare.git && cd PetCare

# 2. Construire et démarrer les 6 services
docker compose up -d --build
```

Mon entrypoint intelligent prend ensuite tout en charge automatiquement : génération des clés JWT RSA (si absentes), vérification de la connexion PostgreSQL, exécution des migrations Doctrine, insertion idempotente des types d'événements par défaut, puis warmup du cache Symfony — sans aucune intervention manuelle de ma part.

2.3 — Entrypoint intelligent (docker/php/entrypoint.sh)

J'ai introduit une variable d'environnement `WORKER_MODE` pour différencier le comportement de mes deux services PHP. Elle m'a permis de résoudre une **race condition** où les services `app` et `worker` lançaient simultanément les migrations au démarrage.

WORKER_MODE=0 — service app (défaut)	WORKER_MODE=1 — service worker
1. Générer les clés JWT (si absentes) 2. Vérifier la connexion PostgreSQL 3. Lancer les migrations Doctrine 4. Warmup du cache Symfony 5. `exec php-fpm`	1. `sleep` — attendre que `app` soit prêt 2. `messenger:consume scheduler_default` (le worker ne touche jamais aux migrations)

2.4 — Variables d'environnement

Je gère toutes les données sensibles via des variables d'environnement, jamais en dur dans le code. Je fournis un fichier `.env.example` documenté dans le dépôt, et mon vrai `.env` est exclu du versionnement Git.

Variable	Description	Exemple
`DATABASE_URL`	DSN PostgreSQL	`postgresql://user:pass@database:5432/petcare`
`JWT_PASSPHRASE`	Passphrase des clés RSA JWT	[générée aléatoirement]
`GOOGLE_CLIENT_ID` / `SECRET`	Identifiants OAuth2 Google	[console Google Cloud]
`MAILER_DSN`	DSN du serveur email	`smtp://mailer:1025` (dev) / Brevo (prod)
`MAILER_FROM_EMAIL`	Adresse expéditrice	`noreply@petcare.fr`
`FRONTEND_URL`	URL du frontend (liens des emails)	`http://localhost:3000`
`CORS_ALLOW_ORIGIN`	Origines CORS autorisées	`^https?://(localhost|127\.0\.0\.1)(:[0-9]+)?$`

---

3. Système d'emails (MailerService)

3.1 — Quatre templates HTML responsive

J'ai centralisé l'envoi de tous les emails transactionnels dans un seul service, `MailerService`. J'ai écrit chaque template en HTML inline (compatible Gmail et Outlook) avec la charte graphique de PetCare.

Template	Déclencheur	Contenu clé
Bienvenue	Inscription (formulaire ET OAuth Google)	Présentation des 4 fonctionnalités, bouton → `/login`
Rappel	N jours avant le RDV, à l'heure exacte	Carte événement colorée, nom de l'animal, date/heure, commentaire, → `/dashboard`
Reset mot de passe	`POST /api/auth/forgot-password`	Lien tokenisé (expiration 1 h), avertissement visuel, → `/reset-password`
Invitation partage	`POST /api/partages`	Nom du propriétaire + animal, badge de rôle, lien d'inscription

3.2 — Configuration selon l'environnement

J'ai conçu le système pour basculer d'un environnement à l'autre par une simple variable `MAILER_DSN`, sans toucher au code.

Environnement	DSN Mailer / Usage
Développement	`smtp://mailer:1025` — Mailpit (UI web : `http://localhost:8025`)
Production	`smtp://...@smtp-relay.brevo.com:587` — relais SMTP Brevo (API externe)
Test (CI)	`null://null` — aucun envoi réel pendant les tests PHPUnit

> Cette séparation **Mailpit (dev) / Brevo (prod)** est une bonne pratique de gestion des environnements : en local je visualise mes emails dans Mailpit sans rien envoyer réellement, et en production Brevo se charge de la distribution aux vraies boîtes.

---

4. Rappels automatiques à l'heure exacte

4.1 — Principe de fonctionnement

J'ai voulu que mes rappels partent à l'**heure exacte du rendez-vous**, N jours avant. Par exemple : un RDV le `30/05/2026 à 12h00` avec `rappelJoursAvant = 2` déclenche l'email le `28/05/2026 à 12h00`, avec une précision à la minute près.

4.2 — Architecture Symfony Scheduler + Messenger

Mon Symfony Scheduler déclenche un message `DailyReminderMessage` toutes les minutes (cron `* * * * *`). Symfony Messenger le consomme via mon service `worker` Docker, appelle `EvenementRepository::findRappelsDuJour()`, envoie l'email via `MailerService::sendReminderEmail()`, puis marque `rappelEnvoye = true` avant de flusher l'EntityManager.

```
Schedule.php (cron * * * * *) → DailyReminderMessage
                    ↓
          DailyReminderMessageHandler
                    ↓
    EvenementRepository::findRappelsDuJour()
                    ↓
       MailerService::sendReminderEmail()
                    ↓
          evenement.rappelEnvoye = true
```

4.3 — Logique de la fenêtre minute

À chaque exécution, je calcule une fenêtre d'une minute `[windowStart, windowEnd[` correspondant à la minute courante. Pour chaque événement candidat (`rappelActif=true`, `rappelEnvoye=false`, date future), je calcule l'heure de rappel comme `dateHeureEvenement − rappelJoursAvant jours`. Si cette heure tombe dans la fenêtre courante, j'envoie l'email.

4.4 — Champ rappelEnvoye — anti-doublon

Situation	Comportement que j'ai implémenté
Rappel envoyé avec succès	`rappelEnvoye = true` → plus jamais redéclenché
Modification de la date de l'événement	`rappelEnvoye = false` → rappel autorisé au nouveau créneau
Modification de `rappelJoursAvant`	`rappelEnvoye = false` → recalcul du créneau
Réactivation de `rappelActif`	`rappelEnvoye = false` → nouvel envoi possible

4.5 — Commande manuelle

J'ai aussi exposé une commande pour déclencher les rappels à la demande (utile en démonstration) :

```
docker compose exec app php bin/console app:send-reminders
```

---

5. Pipeline CI/CD GitHub Actions

5.1 — Déclencheurs

Ma pipeline se déclenche automatiquement à chaque `push` sur `main` et `develop`, ainsi qu'à l'ouverture ou la mise à jour d'une Pull Request. Mes deux branches principales sont ainsi protégées par ces vérifications.

5.2 — Les 3 jobs de la pipeline

Job	Étapes	Statut
Job 1 — PHPUnit (PHP 8.4)	1. `cp .env.test .env` → 2. `composer install` → 3. `composer audit` (0 advisory) → 4. openssl → clés JWT RSA → 5. `phpunit tests/Unit` (57) → 6. `phpunit tests/Functional` (110)	Passing
Job 2 — PHP Lint (PHP 8.4)	`php -l` sur tous les fichiers `src/**/*.php` — détecte toute erreur de syntaxe avant déploiement	Passing
Job 3 — Frontend (Node 20 & 22)	1. `npm install` → 2. `npm audit --audit-level=high` (0 vuln.) → 3. `npm test` — Vitest (26) → 4. `npm run build` → 5. vérification de `dist/`	Passing

> J'ai intégré les scans de sécurité des dépendances directement dans la CI (`composer audit` côté PHP, `npm audit` côté Node), ce qui couvre la catégorie **OWASP A06 — Composants vulnérables**. Toute nouvelle CVE dans une dépendance fait échouer le build, ce qui m'oblige à la traiter avant de fusionner.

5.3 — Stratégie de déploiement et de mise en production

Dans mon contexte pédagogique, la stratégie que j'ai retenue est le **déploiement manuel via Docker Compose** sur un serveur cible : je clone le dépôt, je copie `.env.example` vers `.env`, je renseigne les variables de production, puis je lance `docker compose up -d --build` — mon entrypoint exécute alors automatiquement les migrations et le seed.

Pour une mise à jour **sans interruption de service**, j'ai étudié la stratégie **Blue/Green** : déployer la nouvelle version en parallèle (Green), basculer le routage nginx une fois qu'elle est prête, puis arrêter l'ancienne (Blue). Je documente cette évolution comme perspective pour une v2. Une alternative serait le **rolling update** (mise à jour progressive conteneur par conteneur).

---

6. Sécurité & conformité RGPD

Ce volet synthétise le chapitre IX — Sécurité (détaillé dans mon rapport OWASP/RGPD du Jalon 5).

6.1 — Mesures OWASP Top 10 mises en place

Risque OWASP	Comment je l'ai traité dans PetCare
A01 — Contrôle d'accès	Firewall Symfony stateless JWT, `access_control` sur `/api`, vérification du propriétaire sur chaque ressource, rôles de partage lecture/écriture
A02 — Défaillances cryptographiques	Mots de passe hachés (algorithme `auto` : bcrypt/Argon2), JWT signés par clés RSA, secrets en variables d'environnement
A03 — Injection	Doctrine ORM avec requêtes paramétrées exclusivement, validation des entrées via Symfony Validator
A05 — Mauvaise configuration	Headers HTTP de sécurité (X-Frame-Options, X-Content-Type-Options...) ajoutés via un EventSubscriber et nginx, CORS restreint
A06 — Composants vulnérables	`composer audit` + `npm audit` exécutés à chaque build CI
A07 — Authentification	JWT, limitation des tentatives de connexion (rate limiter), reset par lien tokenisé expirant

6.2 — Conformité RGPD

Parce que mon application manipule des données personnelles, j'ai implémenté deux droits clés de l'utilisateur.

- Le **droit à la portabilité** passe par l'endpoint `GET /api/me/export`, qui exporte l'ensemble des données de l'utilisateur au format JSON.
- Le **droit à l'effacement** passe par l'endpoint `DELETE /api/me`, qui supprime le compte et toutes les données associées (animaux, événements, partages).

6.3 — Maintenance de sécurité continue

> Lors de ma revue finale, mes scans CI ont détecté de nouvelles CVE publiées dans des dépendances transitives (`guzzlehttp/guzzle` et `guzzlehttp/psr7` côté PHP, `form-data` côté Node — failles d'injection CRLF). Je les ai corrigées en mettant à jour ces paquets vers leurs versions patchées (**guzzle 7.12.3, psr7 2.12.3**). Après mise à jour, `composer audit` et `npm audit` ne remontent plus aucune vulnérabilité. Cet épisode illustre concrètement l'intérêt d'avoir intégré les scans de sécurité à ma CI.

---

7. Vérification de conformité au CDC technique

J'ai repris point par point les exigences du cahier des charges technique pour vérifier que mon livrable final les satisfait toutes.

Exigence du CDC technique	Statut	Livrable / Référence
Code source final taggé « release » (v1.0)	Conforme	github.com/Nastry69/PetCare — tag `v1.0`
docker-compose.yml — lancement en une commande	Conforme	Section 2 — `docker compose up -d --build`
Entrypoint automatisé (migrations, JWT, seed)	Conforme	`docker/php/entrypoint.sh` — Section 2.3
Instructions de déploiement (README)	Conforme	README.md + Section 8
Stratégie de mise en production documentée	Conforme	Section 5.3 (Blue/Green)
Toutes les fonctionnalités V1 implémentées et testées	Conforme	Section 1.1 — F1 à F6
Tests automatisés (167 tests, 336 assertions, 0 échec)	Conforme	Section 5.2 + Jalon 5
Base de données relationnelle (Doctrine ORM)	Conforme	PostgreSQL 16 — Jalon 3
API externe intégrée	Conforme	OAuth Google + Brevo SMTP
Containerisation Docker	Conforme	Section 2 — 6 services
Intégration continue (CI)	Conforme	Section 5 — 3 jobs GitHub Actions
Scans de sécurité des dépendances en CI	Conforme	`composer audit` + `npm audit` — Section 5.2 / 6.3
Sécurité OWASP Top 10	Conforme	Section 6 + rapport OWASP/RGPD
Conformité RGPD (export + suppression)	Conforme	Section 6.2
Stratégie de branches Git (main / develop / feature)	Conforme	GitFlow simplifié — Jalon 2
Documentation complète	Conforme	Ce document + README + `Docs/`

---

8. Documentation & guide d'utilisation

8.1 — Procédure d'installation (README)

J'ai rédigé un README complet pour que n'importe quel développeur — ou le jury — puisse déployer mon application sans avoir à me poser de question.

Section du README	Contenu
Installation rapide	2 commandes : `git clone` + `docker compose up`
Services Docker	Tableau des 6 services avec ports et rôles
Variables d'environnement	Tableau complet avec descriptions et exemples
Endpoints API	Tableau des routes avec méthodes et authentification
Configuration Brevo (prod) & OAuth Google	Étapes pour configurer l'envoi d'emails et les credentials Google
Commandes utiles	Logs, shell conteneur, cache clear, migrations, rappels manuels

8.2 — Guide rapide pour tester l'application

Pour permettre une prise en main immédiate lors de la soutenance, voici le parcours que je recommande.

1. D'abord, accéder au frontend sur `http://localhost:3000`, puis créer un compte via *Inscription* (ou se connecter avec Google) : je reçois alors un email de bienvenue, visible dans Mailpit sur `http://localhost:8025`.
2. Ensuite, ajouter un animal avec sa photo, puis créer un événement de soin associé à un rappel.
3. On peut alors partager la fiche de l'animal avec une autre adresse email, en choisissant un rôle lecture ou écriture.
4. Enfin, tester la conformité RGPD depuis les *Paramètres* : exporter mes données, puis, si je le souhaite, supprimer mon compte.

8.3 — Environnements de déploiement

Environnement	Configuration
Développement (local)	`APP_ENV=dev`, Mailpit, base PostgreSQL Docker, debug activé
Test (CI)	`APP_ENV=test`, base SQLite en mémoire, mailer `null://null`
Production	`APP_ENV=prod`, debug désactivé, Brevo SMTP, secrets en variables d'environnement, HTTPS

---

9. Bilan & perspectives

9.1 — Respect du planning

J'ai respecté l'ensemble des six échéances mensuelles du projet.

Jalon	Livrable	Échéance	Statut
Jalon 1 — Janvier	Cahier des charges fonctionnel	31/01/2026	Livré
Jalon 2 — Février	Méthodologie & conception UI/UX	28/02/2026	Livré
Jalon 3 — Mars	Modélisation de la base de données	31/03/2026	Livré
Jalon 4 — Avril	Conception, architecture & UML	30/04/2026	Livré
Jalon 5 — Mai	Développement, sécurité & tests	29/05/2026	Livré
Jalon 6 — Juin	Déploiement & livrable final	30/06/2026	Livré

9.2 — Difficultés rencontrées et solutions

Difficulté rencontrée	Solution que j'ai adoptée
Race condition app/worker au démarrage Docker (migrations lancées deux fois)	Variable `WORKER_MODE` dans l'entrypoint pour différencier les deux comportements de démarrage
Précision des rappels (minute exacte vs jour entier)	Logique de fenêtre glissante minute par minute dans `findRappelsDuJour()` avec cron `* * * * *`
CVE actives dans Symfony détectées par `composer audit` en CI	Mise à jour vers Symfony 7.4.13 — 0 CVE, tests toujours au vert
Nouvelles CVE (guzzle/psr7, form-data) à la revue finale	Mise à jour des dépendances transitives vers leurs versions patchées — audits de nouveau au vert
Tests fonctionnels échouant en CI (FRONTEND_URL manquant)	Ajout de `FRONTEND_URL` dans `.env.test` — variable utilisée par MailerService
Compatibilité des emails HTML (Gmail, Outlook)	CSS inline et structure table-based dans les templates
Clés JWT absentes au premier démarrage et en CI	Génération automatique via openssl dans l'entrypoint et dans le job PHPUnit

9.3 — Évolutions envisageables (v2)

Si je devais poursuivre le projet, plusieurs pistes me paraissent intéressantes.

- Je développerais d'abord une **application mobile native** (React Native ou Flutter), d'autant que mon API REST est déjà prête à être consommée, et j'y ajouterais des **notifications push** en complément des emails de rappel.
- J'imagine ensuite un **mode vétérinaire** offrant un accès en lecture à l'historique complet d'un animal, accompagné de la numérisation des documents vétérinaires (ordonnances, carnet de vaccination) et de statistiques avancées (historique de vaccination, courbes de poids).
- Enfin, côté exploitation, j'automatiserais le **déploiement Blue/Green** via GitHub Actions et DockerHub, et je mettrais en place une **supervision centralisée** (Sentry ou Datadog) pour détecter les incidents en temps réel (OWASP A09).

Conclusion

J'ai désormais finalisé mon projet **PetCare dans sa version 1.0**. L'application répond à l'ensemble des exigences que j'avais définies dans mon cahier des charges fonctionnel et respecte toutes les contraintes techniques imposées par le CDC technique de l'IPSSI.

Sur le plan technique, ce projet m'a permis de mettre en œuvre une architecture **API REST moderne (Symfony 7.4 / React 19)** conteneurisée via Docker, avec une couverture de tests solide — **167 tests PHPUnit** (57 unitaires + 110 fonctionnels, 336 assertions) et **26 tests frontend Vitest** — une sécurité conforme à l'OWASP Top 10, et un système de rappels automatiques précis à la minute.

Sur le plan méthodologique, le découpage en 6 jalons mensuels m'a permis de progresser de façon structurée, en livrant un artefact validable à chaque étape et en évitant l'effet « tunnel ». L'utilisation de Git et de GitHub Actions m'a imposé une rigueur professionnelle directement réutilisable en entreprise.

Ce projet fil rouge constitue pour moi une démonstration concrète de ma capacité à **conduire un projet applicatif complet**, de l'analyse du besoin jusqu'à la mise en production, en mobilisant l'ensemble des compétences attendues d'un Concepteur Développeur d'Applications.

> **Jalon 6 — Livrable Final — PetCare v1.0**
> Tristan DZIOCH — Formation CDA IPSSI — Juin 2026